

UNIVERSIDADE FEDERAL DE ITAJUBÁ

Letícia dos Santos Matos

Simulador Virtual de Estação de Tratamento de Efluentes para Análise de Segurança Cibernética

Implementação do Gêmeo Digital e Arquitetura de Controle

Documentação técnica referente ao desenvolvimento do ambiente de simulação e controle.

Este documento consolida a arquitetura Open-PLC, FUXA e Python, servindo como base experimental para a execução das atividades previstas no Plano de Trabalho.

Orientador: Prof. Otávio de Souza Martins
Gomes

Itajubá - MG
Novembro de 2025

1 Introdução

A convergência entre a Tecnologia da Informação (TI) e a Tecnologia de Operação (TO), impulsionada pela Indústria 4.0, trouxe avanços significativos em eficiência e monitoramento remoto para sistemas industriais. No entanto, essa conectividade ampliada expôs Sistemas de Controle Industrial (ICS) a novos vetores de ameaças cibernéticas. Infraestruturas críticas, como Estações de Tratamento de Efluentes (ETE), tornaram-se alvos potenciais de ataques que podem resultar não apenas em prejuízos econômicos, mas em danos ambientais e riscos à saúde pública.

A análise de vulnerabilidades e o desenvolvimento de estratégias de defesa em ambientes industriais reais são tarefas complexas e perigosas, devido ao risco de interrupção de serviços essenciais ou danos físicos aos equipamentos. Neste contexto, o desenvolvimento de simuladores virtuais e "Gêmeos Digitais" (Digital Twins) apresenta-se como uma abordagem segura e economicamente viável para a pesquisa em segurança cibernética ofensiva e defensiva.

Este trabalho propõe o desenvolvimento de um simulador virtual de uma ETE, utilizando uma arquitetura baseada em ferramentas de código aberto e protocolos padrão da indústria. A solução integra a lógica de controle, a supervisão e a simulação física do processo, criando um ambiente realista (Testbed) para validação de ataques e contramedidas.

1.1 OpenPLC na Pesquisa de Segurança

O componente central da arquitetura de controle proposta é o **OpenPLC**. Trata-se da primeira plataforma de Controlador Lógico Programável (CLP) de código aberto totalmente funcional, compatível com a norma internacional IEC 61131-3.

Diferente de controladores proprietários, que funcionam como "caixas-pretas", o OpenPLC permite acesso total ao seu código-fonte e arquitetura interna. Isso o torna uma ferramenta valiosa para a pesquisa em segurança cibernética, permitindo a análise detalhada de como malwares podem infectar a lógica de controle (como no caso do Stuxnet), como funcionam ataques de injeção de comandos via Modbus e como implementar rotinas de segurança diretamente no nível do controlador. No presente projeto, o OpenPLC executa a lógica de estados finitos responsável pelo gerenciamento das bombas, válvulas e intertravamentos de segurança da ETE.

1.2 Supervisão e Operação com FUXA

Para a camada de Interface Homem-Máquina (IHM) e aquisição de dados, utiliza-se o **FUXA** (Web-based Process Visualization). O FUXA é um software SCADA (*Supervisory Control and Data Acquisition*) moderno e de código aberto, que permite a criação de sinóticos operacionais acessíveis via navegador web.

A escolha do FUXA justifica-se pela sua capacidade nativa de comunicação através do protocolo Modbus TCP/IP, amplamente utilizado na indústria, e pela facilidade de integração com o OpenPLC. No simulador desenvolvido, o FUXA atua como a console do operador, permitindo o envio de comandos de partida, parada e reset, bem como a visualização em tempo real dos níveis dos tanques e estados dos atuadores, replicando a experiência de uma sala de controle real.

1.3 Objetivos do Trabalho

O objetivo principal é desenvolver um simulador virtual de uma ETE funcionalmente fiel para análise de cibersegurança. O projeto visa implementar uma lógica de controle robusta no OpenPLC, criar um Gêmeo Digital em Python para simulação física e integrar os componentes via Modbus TCP/IP, estabelecendo um *Testbed* validado para futuros testes de penetração e defesa em redes industriais.

1.4 Arquitetura do Sistema

O sistema é composto por três camadas interconectadas via protocolo **Modbus TCP/IP**:

- **Controlador (OpenPLC):** Executa a lógica de controle (Linguagem ST), intertravamentos e máquina de estados.
- **Supervisório (FUXA SCADA):** Interface Homem-Máquina (IHM) para operação, visualização de alarmes e monitoramento de níveis.
- **Gêmeo Digital (Python):** Um script que simula a física do processo (hidráulica e sensores), permitindo testar o sistema sem riscos físicos.

2 Diagrama de Instrumentação (P&ID)

A planta é composta por três tanques em série, controlados por bombas de transferência e válvulas de bloqueio.

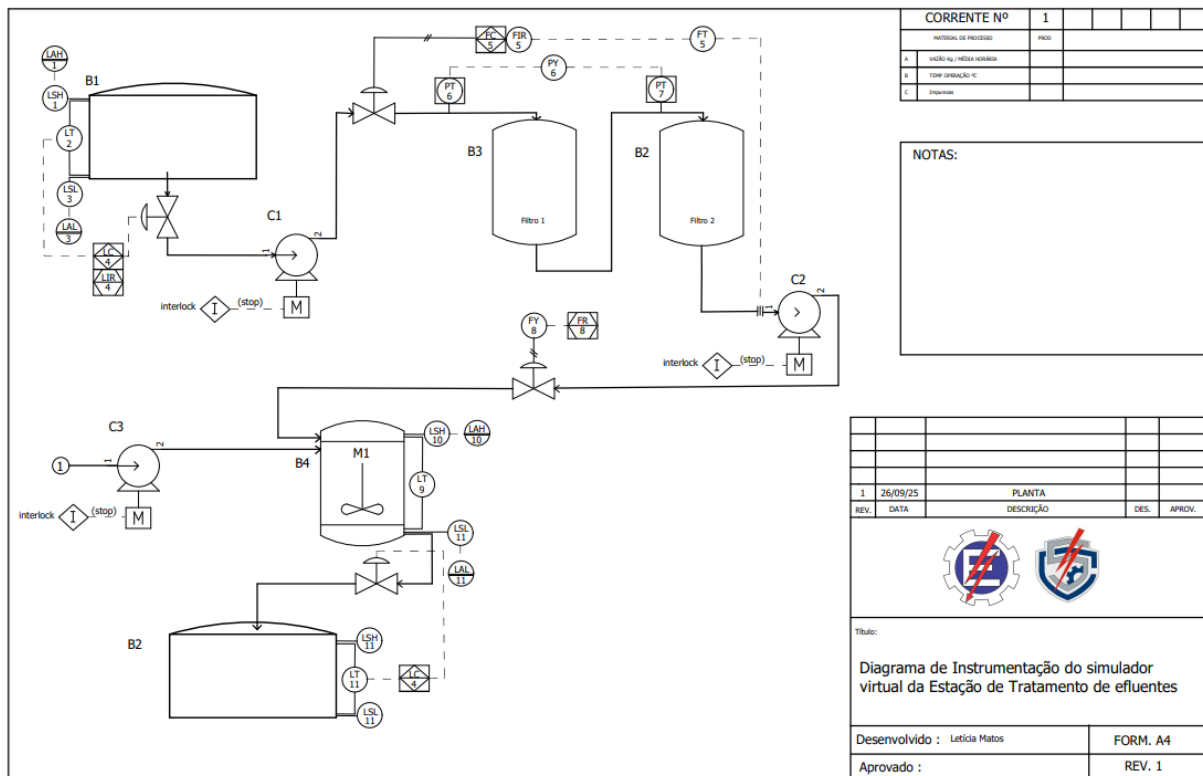


Figura 1: Diagrama P&ID da Estação de Tratamento.

O processo segue o fluxo operacional descrito abaixo:

1. **Armazenamento (B1):** O efluente bruto inicia no tanque **B1**.
2. **Filtração:** A bomba **C1** succiona o líquido e o direciona através dos estágios de filtração **B3** e **B2**.
3. **Transferência Intermediária:** A bomba **C2** transfere o efluente filtrado para o tanque de processo.
4. **Mistura (M1/B4):** No tanque **M1**, ocorre a homogeneização através do agitador (controlado pela lógica da saída C3).
5. **Armazenamento Final (B2):** Após o tratamento, a válvula de saída libera o efluente tratado para o tanque final **B2**.

3 Mapeamento de Entradas e Saídas (I/O List)

A definição dos endereços de memória é a base para a comunicação entre o controlador lógico (OpenPLC) e a interface supervisória (FUXA).

A seguir, apresenta-se a lista de variáveis conforme configurado no projeto, relacionando o endereço físico IEC 61131-3 com o endereço Modbus utilizado na rede.

3.1 Variáveis Analógicas e de Estado (Registradores)

Os níveis dos tanques e o estado da máquina utilizam registradores de 16 bits (Holding Registers). No FUXA, estes são endereçados na faixa 40000x.

Tabela 1: Lista de Registradores (Holding Registers)

Variável OpenPLC	Endereço IEC	Endereço FUXA	Descrição
xiNivel_T1	%QW2	400002	Transmissor de Nível Tanque 1
xiNivel_T2	%QW4	400004	Transmissor de Nível Tanque 2
xiNivel_T3	%QW6	400006	Transmissor de Nível Tanque 3
Estado_Sistema	%MW8	400008	Variável de Controle de Estado

3.2 Entradas Digitais (Comandos)

As variáveis booleanas de comando, acionadas pelos botões da interface, estão mapeadas nos primeiros endereços de memória (%QX0.x).

Tabela 2: Comandos de Operação (Coils)

Variável OpenPLC	Endereço IEC	Endereço FUXA	Função
xiBotao_Start	%QX0.0	1	Iniciar Ciclo (NA)
xiBotao_Stop	%QX0.1	2	Parar/Pausar Ciclo (NF)
xiEmergencia	%QX0.2	3	Parada de Emergência (NF)
xiBotao_Reset	%QX2.7	24	Reset de Falhas (NA)

3.3 Saídas Digitais (Atuadores)

Os motores e válvulas estão alocados majoritariamente no Byte 1 (%QX1.x). O FUXA realiza a leitura destes endereços para indicar o status (ligado/desligado) na tela.

Tabela 3: Atuadores e Equipamentos (Coils)

Variável OpenPLC	Endereço IEC	Endereço FUXA	Equipamento
xqC1_Motor	%QX1.0	9	Bomba de Entrada (P-01)
xqC2_Motor	%QX1.1	10	Bomba de Transferência (P-02)
xqC3_Motor	%QX1.2	11	Misturador (M-01)
xqV1	%QX1.3	12	Válvula de Saída T1
xqV2	%QX1.4	13	Válvula de Entrada Filtro
xqV3	%QX1.5	14	Válvula de Entrada T2
xqV4	%QX1.6	15	Válvula de Saída T2
xqMisturador_T2	%QX2.6	23	Indicador Visual Misturador

3.4 Flags de Processo e Sinalização

Variáveis calculadas internamente pelo CLP para lógica de controle e alarmes.

Tabela 4: Flags de Status e Alarmes (Coils)

Variável OpenPLC	Endereço IEC	Endereço FUXA	Descrição
xqSistemaAtivo	%QX1.7	16	Indica sistema em operação
T1_Vazio	%QX2.0	17	Sensor Nível Baixo T1
T1_Cheio	%QX2.1	18	Sensor Nível Alto T1
T2_Vazio	%QX2.2	19	Sensor Nível Baixo T2
T2_Cheio	%QX2.3	20	Sensor Nível Alto T2
T3_Vazio	%QX2.4	21	Sensor Nível Baixo T3
T3_Cheio	%QX2.5	22	Sensor Nível Alto T3
xqLampada_Falha	%QX3.0	25	Lâmpada Indicadora de Falha

3.5 Considerações de Segurança

A exposição direta destes endereços de memória através do protocolo Modbus TCP, que carece de criptografia e autenticação nativas, constitui a principal superfície de ataque do sistema simulado. Um atacante com acesso à rede poderia, por exemplo, injetar o valor ‘FALSE’ (0) no endereço de Coil 2 (Emergência) para causar uma parada da planta, ou sobrescrever o Holding Register 4 (Nível T2) para forçar o transbordamento do tanque.

4 Lógica de Controle (OpenPLC)

A lógica foi desenvolvida em **Texto Estruturado (ST)** seguindo a norma IEC 61131-3. O controle é baseado em uma Máquina de Estados Finitos com 7 estados distintos para garantir robustez e facilidade de manutenção.

4.1 Lógica de Retenção de Estado e Falha Segura

Uma das principais características do projeto é a estratégia de recuperação de falhas.

- **Problema:** Em processos físico-químicos, interrupções abruptas podem causar sedimentação de resíduos, comprometendo a qualidade se o processo apenas "continuar" de onde parou.
- **Solução:** Adotou-se a **Retenção de Estado com Reinício de Fase**. Se houver uma parada (Stop ou Emergência), o PLC memoriza qual etapa estava sendo executada (ex: Mistura). Ao retomar, o sistema volta para essa etapa, mas reinicia o temporizador da fase.
- **Justificativa:** Isso garante a homogeneidade da mistura e a segurança do processo (Fail-Safe), evitando que efluentes mal tratados avancem.

4.2 Código Fonte Completo (Structured Text)

Abaixo apresenta-se a implementação integral da lógica de controle no OpenPLC. O código está estruturado em blocos funcionais para garantir a varredura correta das entradas antes da execução da máquina de estados.

```
1 (* ===== *)
2 (* BLOCO 0: SNAPSHOT (MEMORIA DE ESTADO) *)
3 (* ===== *)
4 (* Salva o estado atual se o sistema estiver operando normalmente *)
5 (* (Estados 0 a 4). Isso permite retomar após falhas. *)
6
7 IF (Estado_Sistema >= 0) AND (Estado_Sistema <= 4) THEN
8     Estado_Salvo := Estado_Sistema;
9 END_IF;
10
11
12 (* ===== *)
13 (* BLOCO 1: LEITURA E MAPEAMENTO DE SENSORES *)
14 (* ===== *)
15 (* Converte entradas analogicas em flags booleanas com histerese *)
16
17 T1_Vazio := (xiNivel_T1 < 5);
18 T1_Cheio := (xiNivel_T1 >= 95);
19
20 T2_Vazio := (xiNivel_T2 < 5);
21 T2_Cheio := (xiNivel_T2 >= 80);
22
23 T3_Vazio := (xiNivel_T3 < 5);
24 T3_Cheio := (xiNivel_T3 >= 80);
25
26 (* Atribui o direta para indicador visual do misturador *)
27 xqMisturador_T2 := xqC3_Motor;
28
29
30 (* ===== *)
31 (* BLOCO 2: SEGURANCA E INTERRAVAMENTOS *)
32 (* ===== *)
33
34 (* 1. Parada de Emergencia (Prioridade Maxima) *)
35 IF NOT xiEmergencia THEN
```

```

36     Falha := TRUE;
37     Estado_Sistema := 5; (* Desvia para Estado de Falha *)
38 END_IF;
39
40 (* 2. Pausa Operacional (Stop) *)
41 (* So ativa se nao estiver em Falha, Pausa ou Reset *)
42 IF (NOT xiBotao_Stop) AND (Estado_Sistema <> 5) AND (Estado_Sistema <>
43     6) AND (Estado_Sistema <> 0) THEN
44     Estado_Sistema := 6; (* Desvia para Estado de Pausa *)
45 END_IF;
46
47 (* ===== *)
48 (* BLOCO 3: MAQUINA DE ESTADOS FINITOS *)
49 (* ===== *)
50
51 CASE Estado_Sistema OF
52
53     (* --- ESTADO 0: INICIALIZACAO / RESET --- *)
54     0:
55         xqSistemaAtivo := FALSE; xqLampada_Falha := FALSE;
56         xqC1_Motor := FALSE; xqC2_Motor := FALSE; xqC3_Motor := FALSE;
57         xqV1 := FALSE; xqV2 := FALSE; xqV3 := FALSE; xqV4 := FALSE;
58
59         (* Reseta temporizadores para garantir ciclo limpo *)
60         tFiltro(IN := FALSE);
61         tTransferencia(IN := FALSE);
62
63         (* Condicao de Partida *)
64         IF xiBotao_Start AND T1_Cheio AND NOT Falha AND xiEmergencia
65         THEN
66             Estado_Sistema := 1;
67         END_IF;
68
69     (* --- ESTADO 1: FILTRAGEM (Recirculacao) --- *)
70     1:
71         xqSistemaAtivo := TRUE;
72
73         (* Inicia timer de 5 segundos *)
74         tFiltro(IN := TRUE, PT := T#5s);
75
76         (* Atuadores: Liga Bomba 1 e Valvula 1 *)
77         xqC1_Motor := TRUE; xqV1 := TRUE; xqV2 := TRUE;
78         xqC2_Motor := FALSE; xqV3 := FALSE;
79
80         (* Transicao por Tempo *)
81         IF tFiltro.Q THEN
82             xqC1_Motor := FALSE; xqV1 := FALSE; xqV2 := FALSE;
83             tFiltro(IN := FALSE); (* Reset do timer *)
84             Estado_Sistema := 2; (* Avanca para Envio *)
85         END_IF;
86
87         (* Protecao contra cavitacao *)
88         IF T1_Vazio THEN Falha:=TRUE; Estado_Sistema:=5; END_IF;
89
90
91     (* --- ESTADO 2: ENVIO T1 -> T2 --- *)

```

```

92 2:
93   xqSistemaAtivo := TRUE;
94   xqC2_Motor := TRUE; (* Liga Bomba 2 *)
95   xqV3 := TRUE;      (* Abre Valvula Entrada T2 *)
96
97   (* Transicao por Nivel *)
98   IF T2_Cheio THEN
99       xqC2_Motor := FALSE; xqV3 := FALSE;
100       Estado_Sistema := 3; (* Avanca para Mistura *)
101   ELSIF T1_Vazio THEN
102       Falha := TRUE;
103       Estado_Sistema := 5;
104   END_IF;
105
106
107   (* --- ESTADO 3: MISTURA (Agitacao T2) --- *)
108 3:
109   xqSistemaAtivo := TRUE;
110   tTransferencia(IN := TRUE, PT := T#4s);
111
112   xqC3_Motor := TRUE; (* Liga Misturador *)
113   xqV4 := FALSE;      (* Mantem saida fechada *)
114
115   IF tTransferencia.Q THEN
116       xqC3_Motor := FALSE;
117       tTransferencia(IN := FALSE);
118       Estado_Sistema := 4; (* Avanca para Esvaziamento *)
119   END_IF;
120
121   IF T2_Vazio THEN Falha:=TRUE; Estado_Sistema:=5; END_IF;
122
123
124   (* --- ESTADO 4: ESVAZIAMENTO T2 -> T3 --- *)
125 4:
126   xqSistemaAtivo := TRUE;
127   xqC3_Motor := FALSE;
128   xqV4 := TRUE; (* Abre Valvula de Saida *)
129
130   (* Fim do Ciclo *)
131   IF T3_Cheio THEN
132       xqV4 := FALSE;
133       Estado_Sistema := 0; (* Retorna ao Inicio *)
134   ELSIF T2_Vazio THEN
135       Falha := TRUE;
136       Estado_Sistema := 5;
137   END_IF;
138
139
140   (* --- ESTADO 5: FALHA (Safety Latch) --- *)
141 5:
142   xqLampada_Falha := TRUE; xqSistemaAtivo := FALSE;
143
144   (* Safety Stop: Desliga tudo *)
145   xqC1_Motor := FALSE; xqC2_Motor := FALSE; xqC3_Motor := FALSE;
146   xqV1 := FALSE; xqV2 := FALSE; xqV3 := FALSE; xqV4 := FALSE;
147
148   (* Reseta timers para forcar reinicio de fase na retomada *)
149   tFiltro(IN := FALSE);

```

```

150     tTransferencia(IN := FALSE);
151
152     (* Reset Manual (Requer acao do operador) *)
153     IF xiBotao_Reset AND xiEmergencia THEN
154         Falha := FALSE; xqLampada_Falha := FALSE;
155         Estado_Sistema := Estado_Salvo; (* Retoma ponto de parada *)
156
157         (* Protecao contra loop infinito *)
158         IF Estado_Sistema >= 5 THEN Estado_Sistema := 0; END_IF;
159     END_IF;
160
161
162     (* --- ESTADO 6: PAUSA (Hold) --- *)
163     6:
164     xqSistemaAtivo := FALSE;
165     (* Desliga saidas, mantem memoria *)
166     xqC1_Motor := FALSE; xqC2_Motor := FALSE; xqC3_Motor := FALSE;
167     xqV1 := FALSE; xqV2 := FALSE; xqV3 := FALSE; xqV4 := FALSE;
168
169     tFiltro(IN := FALSE);
170     tTransferencia(IN := FALSE);
171
172     (* Retomada *)
173     IF xiBotao_Start AND xiEmergencia THEN
174         Estado_Sistema := Estado_Salvo;
175     END_IF;
176
177     (* Abortar *)
178     IF xiBotao_Reset THEN Estado_Sistema := 0; END_IF;
179
180 END_CASE;

```

Listing 1: Lógica de Controle Completa (Main.st)

5 Supervisório (FUXA SCADA)

A Interface Homem-Máquina (IHM) foi desenvolvida no software FUXA e estruturada em duas telas principais de navegação, separando a operação do processo do diagnóstico de falhas. A comunicação com o CLP ocorre via Modbus TCP/IP com ciclo de atualização de 500ms.

5.1 Tela de Visão Geral (Operação)

A tela principal ("Overview") apresenta o sinótico do processo, permitindo a visualização espacial dos tanques, tubulações e atuadores. É nesta tela que o operador realiza o controle ativo do sistema.

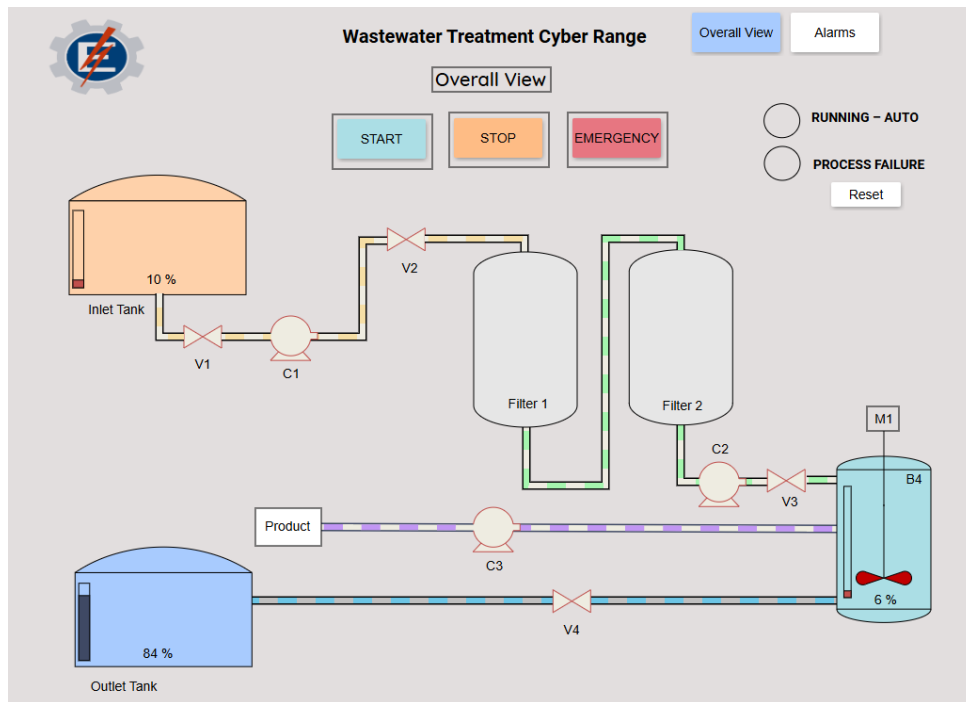


Figura 2: Interface de Operação: Visão Geral do Processo.

5.2 Tela de Alarmes e Eventos

A segunda tela é dedicada ao registro do estado das saídas e monitoramento de falhas.

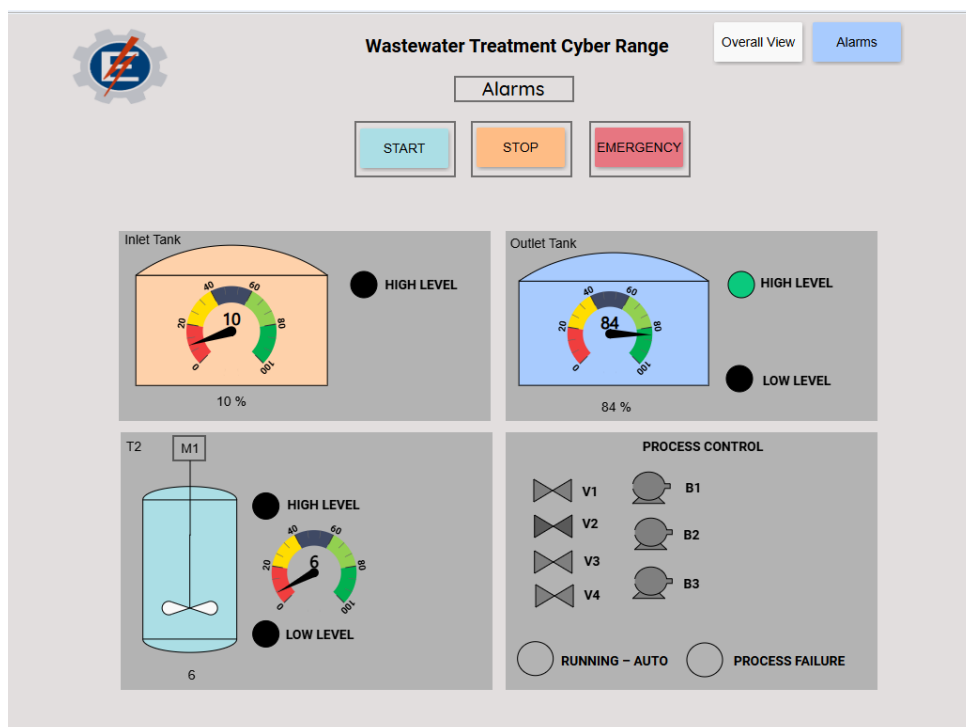


Figura 3: Interface de Diagnóstico: Histórico de Alarmes.

6 Gêmeo Digital (Simulador em Python)

Para validar a lógica de controle sem riscos aos equipamentos físicos, foi desenvolvido um Gêmeo Digital utilizando a linguagem Python. Este software atua como uma camada de abstração de hardware, simulando o comportamento hidráulico da planta em tempo real.

6.1 Funcionamento do Simulador

O script opera em um laço infinito (*Infinite Loop*) com um ciclo de atualização definido pela variável VELOCIDADE_SIMULACAO. A cada iteração, o sistema realiza três etapas críticas:

1. **Leitura de Atuadores:** Através da biblioteca `pymodbus`, o script lê o estado das bobinas (%QX) do OpenPLC para identificar quais bombas e válvulas estão acionadas.
2. **Cálculo Físico:** Com base nos atuadores ligados, o script calcula a variação de volume nos tanques. Foi implementada uma lógica de proteção de simulação (Safety Interlock) que impede que os níveis virtuais desçam abaixo de 10% durante a operação, evitando que o PLC entre em estado de falha devido a atrasos de comunicação (latência).
3. **Escrita de Sensores:** Os novos níveis calculados são escritos de volta nos registradores de retenção (%MW) do OpenPLC, fechando a malha de controle.

6.2 Código Fonte do Simulador

Abaixo apresenta-se o código completo utilizado para a simulação da física do processo.

```
1 import time
2 from pymodbus.client import ModbusTcpClient
3
4 # --- CONFIGURACOES ---
5 PLC_IP = "127.0.0.1"
6 PLC_PORT = 502
7 VELOCIDADE_SIMULACAO = 0.2
8 TAXA_ENCHIMENTO = 6
9
10 # --- ENDEREÇOS ---
11 REG_T1 = 2
12 REG_T2 = 4
13 REG_T3 = 6
14 REG_ESTADO = 8
15
16 client = ModbusTcpClient(PLC_IP, port=PLC_PORT)
17
18 def iniciar_simulacao():
19     if not client.connect():
20         print(f"Erro: Falha ao conectar em {PLC_IP}")
21         return
22
23     print(f"Simulador Conectado. Protecoes de Nivel Ativas.")
24     print("-" * 70)
25
```

```

26 # Condições Iniciais: T1 Cheio
27 nivel_t1, nivel_t2, nivel_t3 = 100, 0, 0
28 client.write_register(REG_T1, nivel_t1)
29 client.write_register(REG_T2, nivel_t2)
30 client.write_register(REG_T3, nivel_t3)
31
32 try:
33     while True:
34         # 1. Ler Dados
35         rr_coils = client.read_coils(0, count=40)
36         rr_regs = client.read_holding_registers(REG_T1, count=7)
37
38         if rr_coils.isError() or rr_regs.isError():
39             time.sleep(1); continue
40
41         bits = rr_coils.bits
42         estado_plc = rr_regs.registers[6]
43
44         # Mapeamento
45         c1_motor = bits[8]
46         c2_motor = bits[9]
47         v1 = bits[11]
48         v3 = bits[13]
49         v4 = bits[14]
50         misturador = bits[22]
51
52         # --- 2. FISICA DA SIMULACAO ---
53
54         # FASE 1: Esvaziamento do T1 (Filtracao)
55         if c1_motor and v1:
56             # Protecao: Nao deixa T1 secar antes da hora
57             if nivel_t1 > 10:
58                 nivel_t1 -= TAXA_ENCHIMENTO
59
60         # FASE 2: Enchimento do T2 (Transferencia)
61         if c2_motor and v3:
62             # T2 enche livremente
63             if nivel_t2 < 100: nivel_t2 += TAXA_ENCHIMENTO
64
65         # FASE 3: Transferencia Final (T2 -> T3)
66         if v4:
67             # T3 enche livremente
68             if nivel_t3 < 100: nivel_t3 += TAXA_ENCHIMENTO
69
70             # So tira agua do T2 se ele estiver acima de 10%.
71             # Isso garante que ele nunca atinja o gatilho de falha
72             (<5%)
73             # enquanto a bomba estiver ligada.
74             if nivel_t2 > 10:
75                 nivel_t2 -= TAXA_ENCHIMENTO
76
77         # Limites
78         nivel_t1 = max(0, min(100, nivel_t1))
79         nivel_t2 = max(0, min(100, nivel_t2))
80         nivel_t3 = max(0, min(100, nivel_t3))
81
82         # 3. Escrever
83         client.write_register(REG_T1, int(nivel_t1))

```

```

83         client.write_register(REG_T2, int(nivel_t2))
84         client.write_register(REG_T3, int(nivel_t3))
85
86     # 4. Display
87     val_v1 = "ON" if v1 else "OFF"
88     val_v3 = "ON" if v3 else "OFF"
89     mix = "ON" if misturador else "OFF"
90     val_v4 = "ON" if v4 else "OFF"
91
92     print(f"\r[Est:{estado_plc}] T1:{int(nivel_t1):02d}% (V1:{
val_v1}) > T2:{int(nivel_t2):02d}% (V3:{val_v3}) Mix:{mix} >(V4:{
val_v4})> T3:{int(nivel_t3):02d}%", end="")
93
94     # 5. Fim de Teste
95     if estado_plc == 0 and nivel_t3 >= 80:
96         print("\n\n" + "="*60)
97         print("SUCESSO! CICLO ENCERRADO.")
98         print("="*60)
99         break
100
101     time.sleep(VELOCIDADE_SIMULACAO)
102
103     except KeyboardInterrupt:
104         print("\nInterrompido.")
105     finally:
106         client.close()
107
108 if __name__ == "__main__":
109     iniciar_simulacao()

```

Listing 2: Script do Simulador Físico (Digital Twin)

7 Conclusão e Trabalhos Futuros

O presente trabalho alcançou o desenvolvimento de um simulador virtual para uma Estação de Tratamento de Efluentes. A integração entre o controlador OpenPLC, o supervisor FUXA e o Gêmeo Digital em Python demonstrou ser uma arquitetura viável e robusta para a representação de processos industriais críticos.

Com a validação funcional concluída, o sistema encontra-se disponível e operacional para análises de segurança cibernética. O ambiente isolado e controlado estabelecido constitui um *Testbed* ideal para a execução de pesquisas ofensivas e defensivas.